# AWS VPC lab — Complete Walkthrough

A public subnet with an app server, a private subnet with a database server, an Internet Gateway, a NAT gateway, and the security groups and route tables that tie it all together. This is the full build, in order

**Region used:** eu-north-1 (Stockholm), AZ eu-north-1a. CIDR: VPC `10.0.0.0/16`, public subnet `10.0.1.0/24`, private subnet `10.0.2.0/24`.

---

## Core concepts,

A VPC is your own private network space, defined by a CIDR block. Subnets are slices of that range, each tied to one Availability Zone. Whether a subnet is "public" or "private" has nothing to do with its name or its CIDR — it's decided entirely by its route table. A subnet is public only if its route table sends internet-bound traffic (`0.0.0.0/0`) to an Internet Gateway.

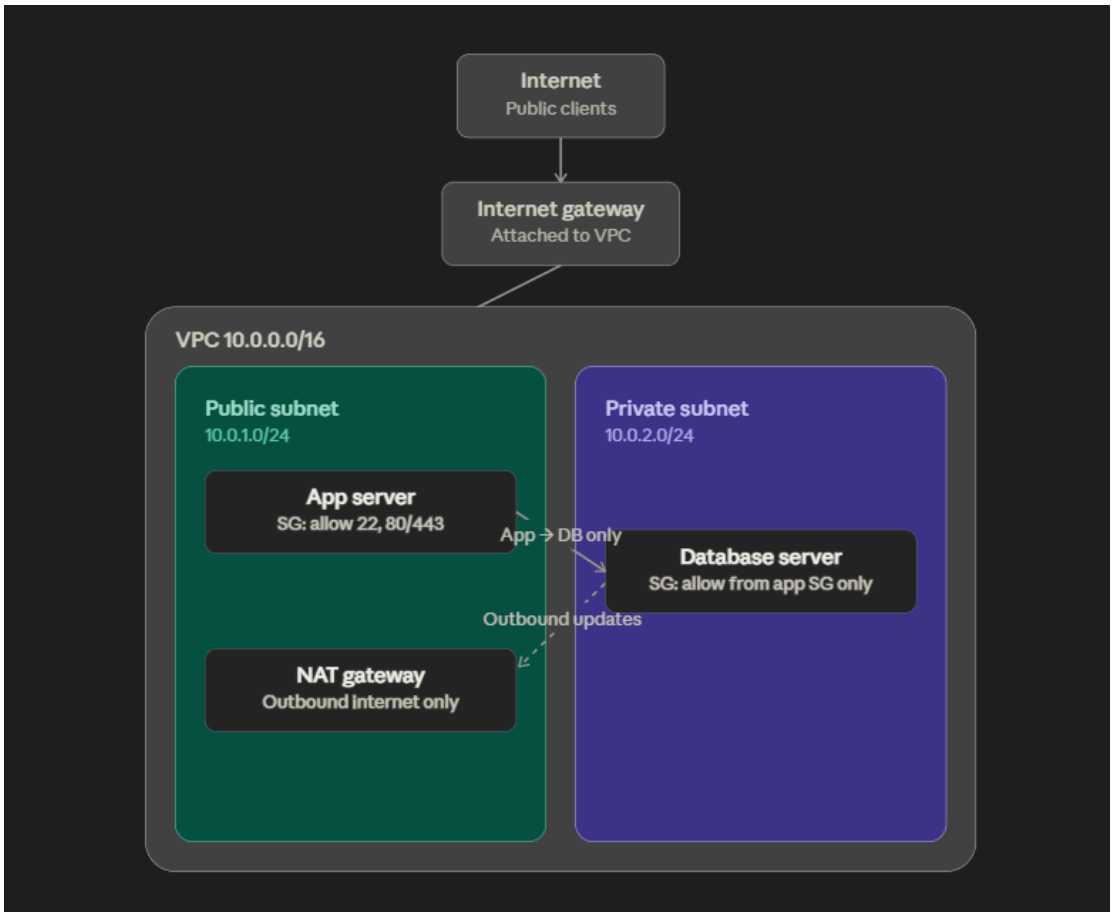
The Internet Gateway does one-to-one translation: it permanently maps one public IP to one private IP, and rewrites addresses on packets crossing that boundary, in both directions. A NAT gateway does many-to-one translation: many private IPs can share one public IP (the NAT gateway's own Elastic IP), but only for connections that start from inside the VPC — it never accepts a new connection coming in from outside. That's the whole reason a database server in a private subnet can fetch OS updates but can never be reached directly from the internet.

Every IP address actually attached to something inside the VPC — every instance, every gateway — is a *private* address from the VPC's own CIDR range. A "public IP" is never literally configured onto an instance; it's a separate address that AWS maps onto the instance's private address at the network edge (the IGW or the NAT gateway).

---

**Created by Harsh Kumar | [HARSH-dev1708]**

# Lab Architecture Diagram




## Part 1 — Create the VPC

1. Go to the VPC console → Your VPCs → Create VPC.
2. Choose "VPC only," name it (e.g. `my-vpc-01`), and set the IPv4 CIDR block to `10.0.0.0/16`.
3. Leave everything else at default and create it.

---

## Part 2 — Create the two subnets

1. VPC console → Subnets → Create subnet, select your VPC.
2. Fill in the first subnet: name `public-subnet`, CIDR `10.0.1.0/24`, Availability Zone `eu-north-1a`.

**Created by Harsh Kumar | [HARSH-dev1708]**

3. Click "Add new subnet" on the same page rather than starting over — fill in the second: name `private-subnet`, CIDR `10.0.2.0/24`, same AZ.
4. Click "Create subnet" once at the bottom to create both together.

*Note on the AZ dropdown:* AWS shows both an AZ name (`eu-north-1a`) and an AZ ID (`eun1-az1`) side by side. Ignore the ID, it's just a way of identifying the same physical location across different AWS accounts. Just pick the same letter for both subnets.

---

## Part 3 — Create and attach the Internet Gateway

1. VPC console → Internet Gateways → Create internet gateway. Name it something traceable, e.g. `my-vpc-01-igw`.
2. After creating it, it'll show as "Detached" — that's expected, creation and attachment are separate steps.
3. Select it, Actions → Attach to VPC, choose your VPC. State should flip to "Attached."

---

## Part 4 — Create the NAT Gateway

1. VPC console → NAT Gateways → Create NAT gateway. Name it (e.g. `my-nat-gateway-01`).
2. **Gotcha:** newer AWS consoles default "Availability mode" to **Regional**, which only asks for a VPC, not a subnet, since it spans all AZs automatically. We want **Zonal** instead, the simpler option tied to one specific subnet. Click Zonal.
3. Once on Zonal, a Subnet field appears — select `public-subnet`. The NAT gateway physically lives in the public subnet even though it exists to serve the private one.
4. Connectivity type: leave as Public.
5. Method of EIP allocation: choose **Automatic** — AWS allocates the Elastic IP for you in the same step, no need to allocate one separately beforehand.
6. Create it, and wait for its status to change from "Pending" to "Available" (takes a couple of minutes) before moving on, since you'll need to select it in a route table next.

---

## Part 5 — Configure the route tables

Two route tables are needed, and each one needs two things done to it: an explicit subnet association, and the actual route.

### Public route table

1. VPC console → Route Tables → Create route table. Name it `public-rt`, pick your VPC.
2. Subnet associations tab → Edit subnet associations → check `public-subnet` → save.
3. Routes tab → Edit routes → Add route. Destination `0.0.0.0/0`. Target → select "Internet Gateway," then **click your actual gateway from the search dropdown that appears — don't type the ID by hand**, typed free text without selecting from the list throws "a valid resource id has to be specified." Save changes.

### Private route table

1. Every VPC automatically gets a default "main" route table the moment it's created — it's easy to mistake this for something you need to create yourself. Either create a genuinely new route table named `private-rt` and use that, or, if you've already renamed the existing main table thinking you created it, that's fine too, just make the association explicit (see below) so it's deliberate rather than an accident.
2. Subnet associations tab → Edit subnet associations → check `private-subnet` → save.
3. Routes tab → Edit routes → Add route. Destination `0.0.0.0/0`. Target → "NAT Gateway" → select your NAT gateway from the dropdown (must show "Available" status first). Save.

---

## Part 6 — Create the security groups

Create `app-sg` first, since `db-sg`'s rule needs to reference it.

### app-sg

1. EC2 console → Security Groups → Create security group. Name `app-sg`, VPC = your VPC.
2. Inbound rules: SSH (port 22), source = **My IP**. Optionally add HTTP/HTTPS from Anywhere-IPv4 if you'll serve web traffic later.
3. Leave outbound rules at default (allow all). Create.

### db-sg

1. Create security group, name `db-sg`, same VPC.

2. Inbound rule 1: Custom TCP, port `5432` (Postgres's default — a placeholder until you pick real database software), source = search and **select** `app-sg` from the dropdown (same click-don't-type rule as the IGW target earlier).
3. Inbound rule 2, **easy to miss**: also add SSH (port 22), source = `app-sg`. Without this, you can SSH into the app server fine but hopping from there into the database server will hang indefinitely — the security group silently drops the port-22 attempt with no error message at all, which is a very recognizable AWS symptom worth remembering.
4. Leave outbound at default. Create.

---

# Part 7 — Launch the EC2 instances

## App server

1. EC2 → Instances → Launch instances. Name `app-server`. AMI: Amazon Linux 2023 (Free tier eligible).
2. Instance type: pick whichever shows the green "Free tier eligible" tag (`t2.micro` or `t3.micro` depending on region).
3. Key pair: Create new key pair, name it `lab-key`, type RSA, format `.pem`. **Save the downloaded file somewhere permanent — it cannot be re-downloaded.**
4. Network settings: VPC = yours, Subnet = `public-subnet`, Auto-assign public IP = **Enable**. Security group: select existing → `app-sg`.
5. Leave storage at default. Launch.

## Database server

1. Launch instances again, name `db-server`, same AMI and instance type.
2. Reuse the same `lab-key` key pair rather than creating a new one — you'll need it for both hops during testing.
3. Network settings: Subnet = `private-subnet`, Auto-assign public IP = **Disable**. Security group: `db-sg`.
4. Launch.

*Reality check, if you're curious:* even if you accidentally enabled auto-assign public IP on the database server, it still wouldn't be reachable from outside. A public IP only does anything if the subnet's route table has a path to the Internet Gateway — `private-subnet`'s doesn't, so an IP sitting there would be functionally dead weight.

---

# Part 8 — Test it

All three commands assume you've downloaded `lab-key.pem` and run `chmod 400 lab-key.pem` on it first (skip chmod on native Windows PowerShell, it'll just no-op).

**Test 1 — laptop to app server (proves the public path works):**

ssh -i lab-key.pem ec2-user@<app-server-public-ip>

Accept the fingerprint prompt with `yes` the first time.

**Test 2 — app server to database server (proves the internal hop works):**

ssh -i lab-key.pem ec2-user@<db-server-private-ip>

If this hangs and eventually fails with "Permission denied," it's because the key file only exists on your laptop, not on the app server. Fix it by copying the key over first:

exit
scp -i lab-key.pem lab-key.pem ec2-user@<app-server-public-ip>:~/
ssh -i lab-key.pem ec2-user@<app-server-public-ip>
chmod 400 lab-key.pem
ssh -i lab-key.pem ec2-user@<db-server-private-ip>

(The cleaner long-term approach is SSH agent forwarding with `ssh-add` + `ssh -A`, but copying the key is simpler to get working on the first try.)

**Test 3 — laptop directly to database server (proves the block is real):**

exit
exit
ssh -i lab-key.pem ec2-user@<db-server-private-ip>

This should just hang with no response at all — no error, no rejection, just silence — because there's genuinely no route for it to travel on. `Ctrl+C` to cancel once you're convinced.

**Bonus — see IGW and NAT actually doing their job:** On the app server, run `curl ifconfig.me` — it should print the app server's own public IP, proof of IGW's one-to-one mapping. Then hop into the database server and run the same command — it'll print a *different* address, the NAT gateway's Elastic IP, proof of the many-to-one NAT translation happening for outbound-only traffic.

---

## Part 9 — Tear it down (stop the billing)

Order matters here.

1. EC2 → Instances → terminate both `app-server` and `db-server`.
2. VPC → NAT Gateways → delete `my-nat-gateway-01`. Wait for it to finish deleting (a few minutes).
3. EC2 → Elastic IPs → once the NAT gateway is fully gone, select the now-unassociated Elastic IP → Actions → Release. This is the step people forget, and the one that quietly keeps billing if skipped.
4. Optional cleanup, no cost impact either way: delete the route tables, detach and delete the Internet Gateway, delete both subnets, delete the security groups, delete the VPC.
5. Confirm: EC2 → Elastic IPs should show nothing, and NAT Gateways / Instances tabs should both be empty.